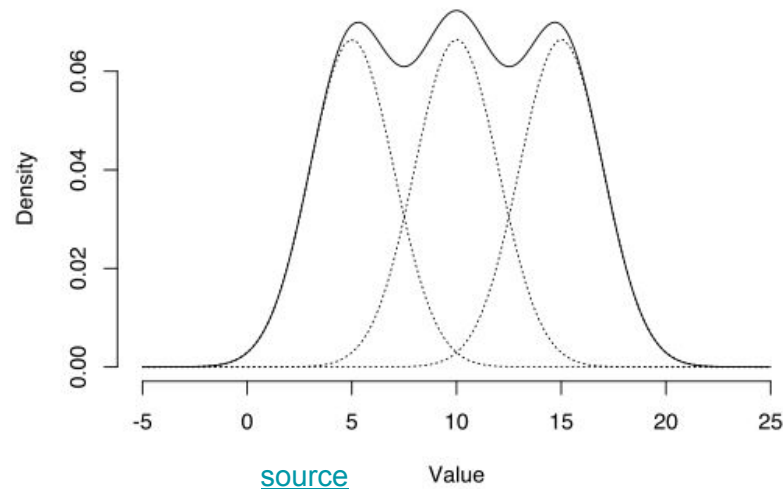


Expectation Maximization: Gaussian Mixture Models

Arin Ghazarian
California State University Long Beach

Mixture Models

- A probabilistic model for representing the presence of subpopulations within an overall population (without requiring that an observed data set should identify the sub-population to which an individual observation belongs)
- A mixture distribution representing the probability distribution of observations in the overall population.



Examples

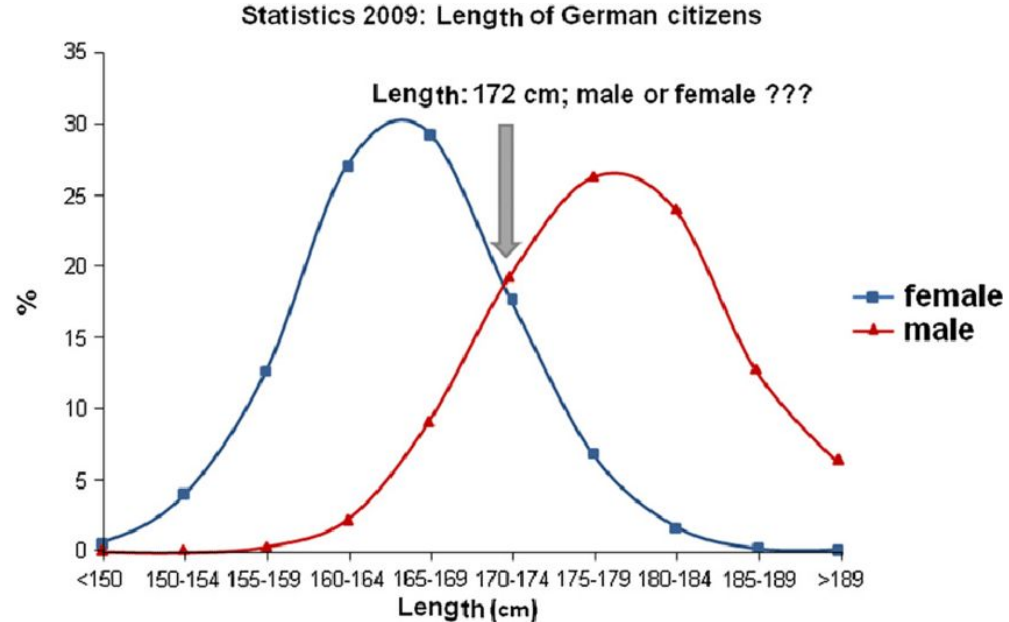
- Financial returns: often behave differently in normal situations and during crisis times
- Housing Prices: a mixture model with K different components, each distributed as a normal distribution with unknown mean and variance, with each component specifying a particular combination of house type/neighborhood.

Gaussian Mixture Model

- A probabilistic model for representing normally distributed subpopulations within an overall population.
- Unsupervised learning
- Very similar to K-means

GMM Example: Human Height

- Human height is typically modeled as a normal distribution for each gender



[source](#)

Gaussian Mixture Models

- Each component is Gaussian
- For a Gaussian mixture model with K components, the k -th component has a mean of μ_k and variance of σ_k
- The mixture component weights are defined as ϕ_k for component C_k , with the constraint that $\sum \phi_i = 1$
- ϕ_i s follow a multinomial distribution

$$p(x) = \sum_{i=1}^K \phi_i \mathcal{N}(x \mid \mu_i, \sigma_i)$$
$$\mathcal{N}(x \mid \mu_i, \sigma_i) = \frac{1}{\sigma_i \sqrt{2\pi}} \exp \left(-\frac{(x - \mu_i)^2}{2\sigma_i^2} \right)$$
$$\sum_{i=1}^K \phi_i = 1$$

Gaussian Mixture Models: Multivariate

$$p(\vec{x}) = \sum_{i=1}^K \phi_i \mathcal{N}(\vec{x} \mid \vec{\mu}_i, \Sigma_i)$$

$$\mathcal{N}(\vec{x} \mid \vec{\mu}_i, \Sigma_i) = \frac{1}{\sqrt{(2\pi)^K |\Sigma_i|}} \exp \left(-\frac{1}{2} (\vec{x} - \vec{\mu}_i)^T \Sigma_i^{-1} (\vec{x} - \vec{\mu}_i) \right)$$

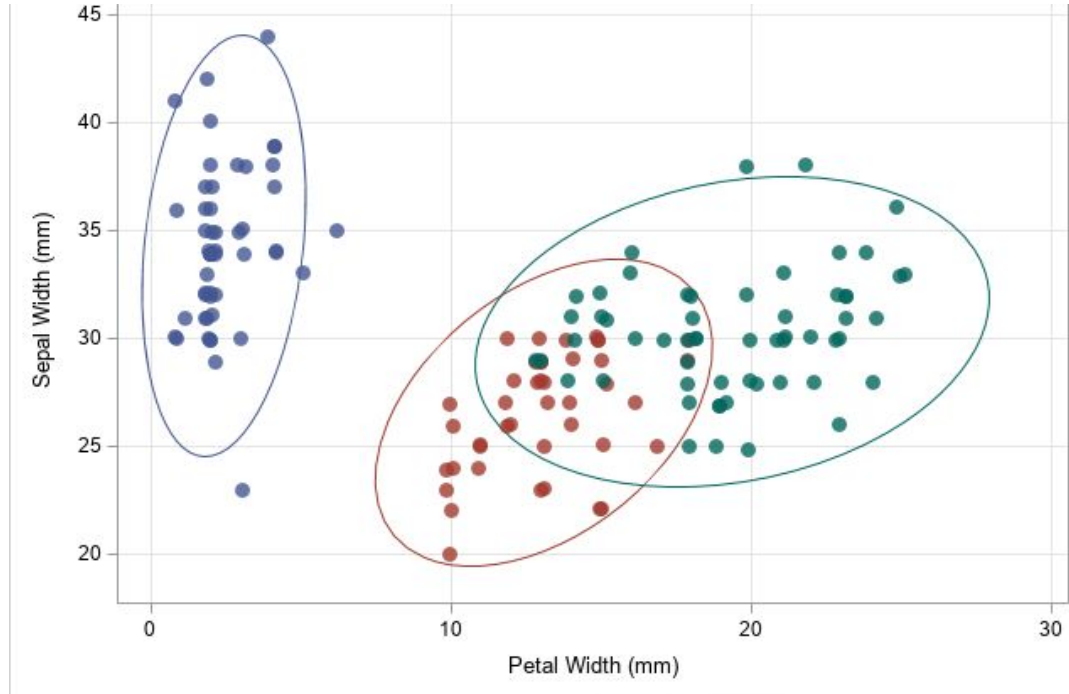
$$\sum_{i=1}^K \phi_i = 1$$

GMM: Universal Approximator

- A probabilistic view of clustering
- Each cluster belongs to a different Gaussian.
- Uses latent variables (the cluster number)
- General approach, can replace Gaussian with other distributions (continuous or discrete)
- GMM is a powerful model
- GMMs are universal approximators of densities (using enough number of Gaussians)
- Optimization is done using the EM algorithm
- GMM is a density estimator

GMM as a Generative Model

- ❖ Each data point is sampled from a generative process:
 - Choose component i with probability $P(y=i)$
 - Generate a random sample according to $\sim N(\mu_i, \Sigma_i)$



Latent Variable

- Variables which are always unobserved are called latent variables, or sometimes hidden variables
- The identity of the component used to generate the data point represent a latent variable or unobservable data (Z)

Marginal Likelihood of the Observed Data

$$p(\mathbf{x}, z) = p(\mathbf{x}|z)p(z)$$

$$p(\mathbf{x}) = \sum_z p(\mathbf{x}, z) = \sum_z p(\mathbf{x}|z)p(z)$$

$$p(\mathbf{x}) = \sum_{k=1}^K p(\mathbf{x}, z = k)$$

$$= \sum_{k=1}^K \underbrace{p(z = k)}_{\pi_k} \underbrace{p(\mathbf{x}|z = k)}_{\mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)}$$

Marginal Likelihood of the Observed Data

- The continuous form will be:

$$L(\boldsymbol{\theta}; \mathbf{X}) = p(\mathbf{X} \mid \boldsymbol{\theta}) = \int p(\mathbf{X}, \mathbf{Z} \mid \boldsymbol{\theta}) d\mathbf{Z} = \int p(\mathbf{X} \mid \mathbf{Z}, \boldsymbol{\theta}) p(\mathbf{Z} \mid \boldsymbol{\theta}) d\mathbf{Z}$$

Maximizing the Marginal Likelihood of the Observed Data

- We need to apply MLE to find the parameters which maximize the log likelihood of the observed data

$$\begin{aligned}\ell(\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) &= \ln p(\mathbf{X} | \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^N \ln p(\mathbf{x}^{(n)} | \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) \\ &= \sum_{n=1}^N \ln \sum_{z^{(n)}=1}^K p(\mathbf{x}^{(n)} | z^{(n)}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) p(z^{(n)} | \boldsymbol{\pi})\end{aligned}$$

- How can we optimize this?
 - Often intractable since Z is unobserved and the distribution of Z is unknown before attaining $\boldsymbol{\mu}, \boldsymbol{\Sigma}$
 - No analytic closed form solution
 - Numerical methods also are difficult and slow

Maximizing the Marginal Likelihood of the Observed Data

- If we knew Z^n , it would be easy to optimize

$$\ell(\boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^N \ln p(\mathbf{x}^{(n)}, z^{(n)} | \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^N \ln p(\mathbf{x}^{(n)} | z^{(n)}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) + \ln p(z^{(n)} | \boldsymbol{\pi})$$

$$\mu_k = \frac{\sum_{n=1}^N 1_{[z^{(n)}=k]} \mathbf{x}^{(n)}}{\sum_{n=1}^N 1_{[z^{(n)}=k]}}$$

$$\Sigma_k = \frac{\sum_{n=1}^N 1_{[z^{(n)}=k]} (\mathbf{x}^{(n)} - \mu_k)(\mathbf{x}^{(n)} - \mu_k)^T}{\sum_{n=1}^N 1_{[z^{(n)}=k]}}$$

$$\pi_k = \frac{1}{N} \sum_{n=1}^N 1_{[z^{(n)}=k]}$$

Expectation maximization (EM)

- The EM algorithm seeks to find the MLE of the marginal likelihood by iteratively applying these two steps until convergence:
 - **E-step:** Compute the posterior probability over z given our current model (how probably each Gaussian generates each datapoint)
 - **M-step:** Assuming that the data really was generated this way, update the parameters of each Gaussian to maximize the probability that it would generate the data it is currently responsible for
- Expectation maximization technique is commonly used to estimate the mixture model's parameters

Expectation maximization (EM)

- The identity of the component used to generate the data point represent a latent variable or unobservable data.
- E-step estimates the expected value for each latent variable
- M-step helps in optimizing them significantly using the Maximum Likelihood Estimation (MLE).
- This process is repeated until a good set of latent values, and a maximum likelihood is estimated

EM vs. K-means

- EM for mixtures of Gaussians is just like a soft version of K-means, with fixed priors and covariance
 - K-means assigns 0s and 1s vs probabilities
- Instead of hard assignments in the E-step, we do soft assignments based on the softmax of the squared Mahalanobis distance from each point to each cluster.
- Each center moved by weighted means of the data, with weights given by soft assignments
 - In K-means, weights are 0 or 1

EM applications

- EM has many applications:
 - Finding GMM parameters
 - Imputing missing data
 - Handwritten character recognition

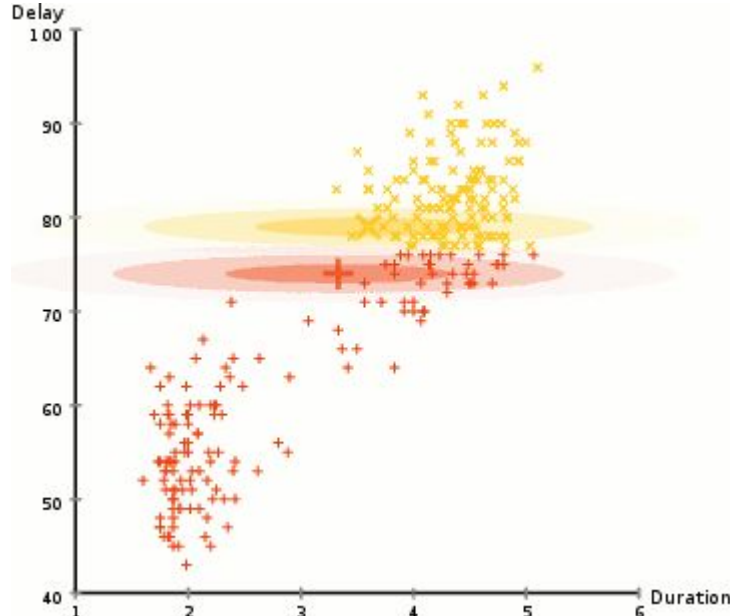
EM: Clustering of Old Faithful Data

- Old Faithful Geyser Data: Waiting time between eruptions and the duration of the eruption for the Old Faithful geyser in Yellowstone National Park,
- A data frame with 272 observations on 2 variables:
 - Eruptions: eruptions duration in mins
 - Waiting: time to next eruption



EM: Clustering of Old Faithful Data

- Starting with a random assignment and unit spheres (note that the axes have different scales), EM quickly adapts to the dataset.
- X axis is the duration of the geyser eruption, Y axis is the delay from the previous eruption.
- [GMM in Python from Scratch: Old Faithful Data](#)



Mixture models in 1-d

- Observations $x_1, \dots, x_n$
 - $K=2$ Gaussians with unknown μ, σ^2
 - estimation trivial if we know the source of each observation

$$\mu_k = \frac{x_1 + x_2 + \dots + x_n}{n_k}$$

$$\sigma_k^2 = \frac{(x_1 - \mu_k)^2 + \dots + (x_n - \mu_k)^2}{n_k}$$



- What if we don't know the source?
- If we knew parameters of the Gaussians (μ, σ^2)
 - can guess whether point is more likely to be a or b

$$P(b|x) = \frac{P(x|b)P(b)}{P(x|b)P(b) + P(x|a)P(a)}$$

$$P(x|b) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu_b)^2}{2\sigma^2}\right)$$

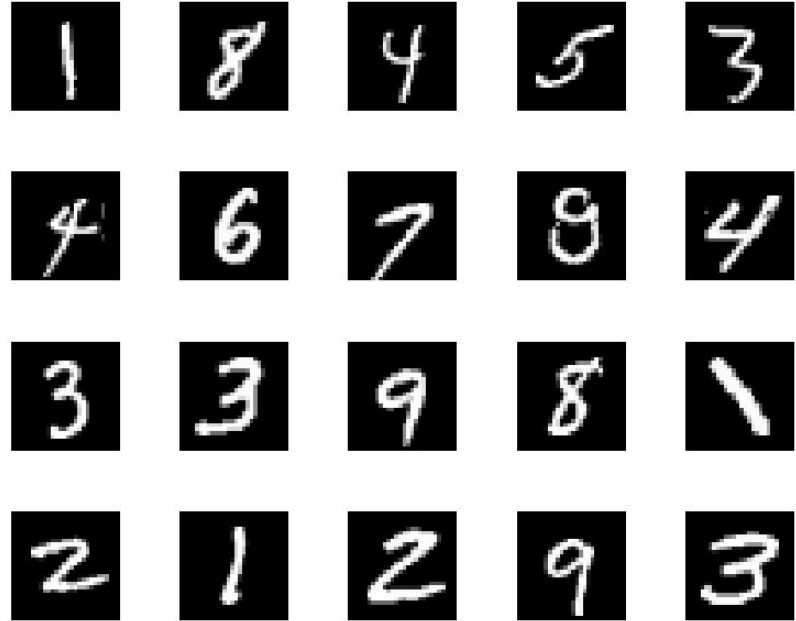


Example Analysis: Mixture of Normals (GMM)

- [EM: Mixture of normals](#)

Example Analysis: Handwritten Digit Recognition

- Input: $N \times N$ black-and-white image that is known to be a scan of a hand-written digit between 0 and 9,
- We don't know which digit is written.
- We can create a mixture model with $K=10$ different components, where each component is a vector of size N^2 of Bernoulli distributions (one per pixel).
- We can train this using expectation-maximization and the output will give us the clusters (Digits)
- The same model could then be used as a classifier, by computing the probability of the new image for each possible component (digit) and returning the digit which had the highest probability



EM: Handwritten Digit Recognition

- [Expectation Maximization for the recognition of handwritten digits in the MNIST dataset](#)